CourseAdvisor Mobile

SYSTEM ARCHITECTURE & DESIGN SPECIFICATION

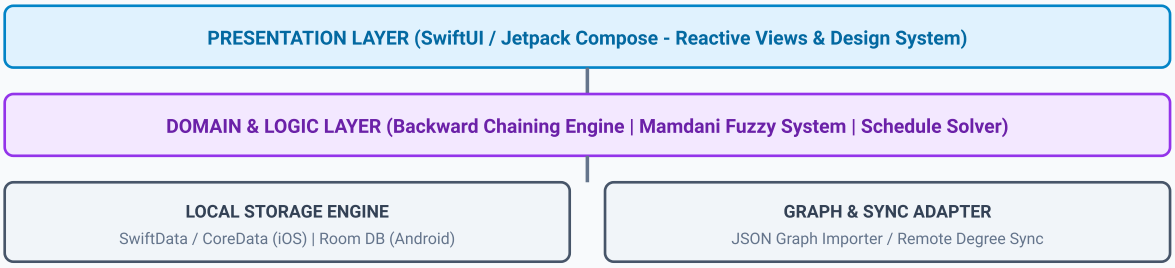
Document ID: ARCH-2026-v2.1
Platform: iOS & Android Native
Classification: Confidential / Official

Executive System Overview

CourseAdvisor is an intelligent, cross-platform mobile academic planning engine designed to calculate optimal, personalized semester class schedules. Unlike traditional static degree-audit tools, CourseAdvisor uses a **Backward Chaining Inference Engine** operating over a Directed Acyclic Graph (DAG) of prerequisite trees, combined with a **Mamdani Fuzzy Inference System**. This dual-engine architecture translates high-level career or degree goals into structured, low-stress academic roadmaps tailored to individual student confidence levels and target course workloads.

1. HIGH-LEVEL ARCHITECTURE & LAYERING

The application follows a strict **Clean Architecture (Unidirectional Data Flow)** pattern on both platforms to isolate complex mathematical and graph evaluation models from platform-specific UI rendering and database storage.



2. ARCHITECTURE CORE PRINCIPLES

DECOUPLING **Independent Algorithmic Core**

The Backward Chaining Graph Traverser and Fuzzy Logic Scoring Engine are designed as zero-dependency modules. They contain no platform UI or framework code, enabling identical logical output across iOS and Android builds.

PERSISTENCE **Offline-First Local Data Engine**

Academic degree trees, course relationships, and completed transcript histories are persisted locally in indexed relational SQLite stores to allow sub-millisecond graph resolution without network dependence.

STATE **Unidirectional State Flow (UDF)**

All UI states are driven by immutable state streams. User input events (e.g., adding a course, adjusting confidence scores) trigger asynchronous pipeline recalculations and emit fresh view models.

SAFETY **Deterministic Verification**

Graph evaluation enforces strict circular dependency detection and topological ordering prior to passing candidate schedules to the Fuzzy Inference System for workload optimization.

3. ALGORITHMIC ENGINE SPECIFICATIONS

The core intelligence of CourseAdvisor relies on two coupled mathematical models: a Goal-Driven Graph Solver and a Dual-Input Fuzzy Inference System.

3.1 Backward Chaining Prerequisite Graph Engine

The academic roadmap is structured as a Directed Acyclic Graph (DAG) $G = (V, E)$, where V represents courses and E represents prerequisite dependencies ($u \rightarrow v$ denotes u requires v).

GRAPH TRAVERSAL & TOPOLOGICAL EVALUATION PIPELINE:

1. **Goal Node Identification:** User selects a target career path or senior capstone course (V_{target}).
2. **Recursive Stack Traversal:** Traverses edges backward from V_{target}. Uncompleted nodes are pushed onto an evaluation stack while ignoring already satisfied courses present in the student's completed transcript set.
3. **Cycle Detection (Tarjan's/Kahn's):** Validates that curriculum graphs contain no circular prerequisite loops.
4. **Topological Tier Assignment:** Groups remaining required courses into ordered prerequisite levels (Tier 0: No pending prerequisites; Tier N: Depends on Tier $N-1$).

3.2 Mamdani Fuzzy Inference System (Workload Scoring)

To eliminate overly congested or stressful academic terms, a Mamdani Fuzzy Inference Engine computes a **Semester Stress Index** and determines maximum recommended credit hours per term.

1. Fuzzification

Converts crisp inputs into linguistic variables using triangular and trapezoidal membership functions.

• **Inputs:** Student Confidence ($C \in [1,10]$), Subject Difficulty ($D \in [1,10]$).

2. Rule Evaluation

Evaluates AND/OR rules against fuzzy matrices.

• *IF Confidence IS Low AND Difficulty IS Hard THEN Risk IS Very High.*
• *IF Confidence IS High AND Difficulty IS Moderate THEN Risk IS Low.*

3. Defuzzification

Applies Centroid Defuzzification to compute crisp workload bounds.

• **Outputs:** Max Recommended Credits (12–18 hrs), Predicted Stress Rating (0–100%).



4. RELATIONAL DATA MODELING & GRAPH SCHEMA

The local database uses an **Adjacency List & Junction Table Structure** optimized for recursive Common Table Expression (CTE) queries in SQLite engines.

Table Name	Primary Entities / Keys	Relationship & Description
	course_id (PK), code, title	Master registry of all university catalog courses and baseline

5. NATIVE MOBILE PLATFORM ARCHITECTURES

CourseAdvisor is optimized natively for both iOS and Android platforms to leverage modern declarative rendering frameworks and native hardware acceleration for graph generation.

IOS PLATFORM Swift & SwiftUI Stack

Architecture Pattern: Clean Architecture + MVVM with Unidirectional Data Flow.

UI Framework: SwiftUI utilizing native `NavigationStack`, interactive graph views via custom `Canvas` layouts, and `Swift Charts` for visual workload distribution analytics.

Persistence Engine: `SwiftData` layered over an encrypted SQLite store. Uses custom `@Model` attributes for efficient graph node traversal.

Concurrency Model: Swift Concurrency (`async/await`, `Actors`) for background calculation of graph nodes without blocking main-thread UI rendering.

ANDROID PLATFORM Kotlin & Compose Stack

Architecture Pattern: MVI (Model-View-Intent) / Clean Architecture driven by Jetpack ViewModel.

UI Framework: Jetpack Compose with Material Design 3 guidelines. Custom layouts built using Compose `Canvas` and custom modifier chains for dynamic node rendering.

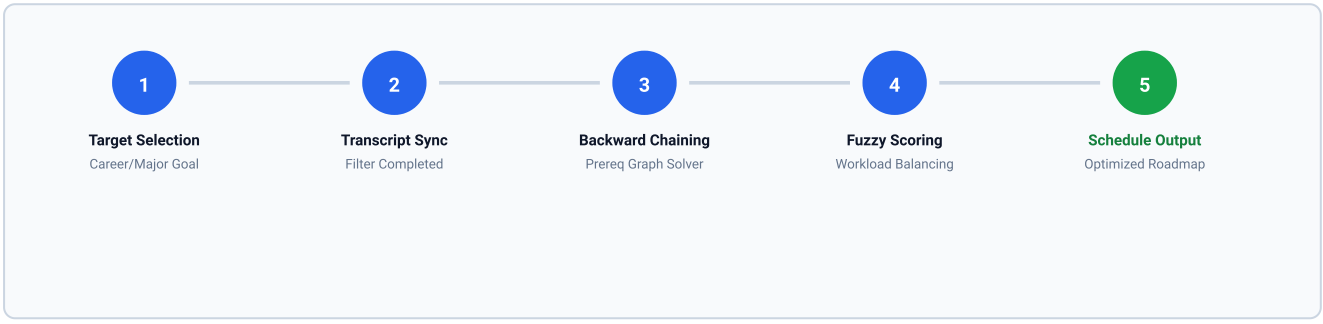
Persistence Engine: `Room Database` using recursive Common Table Expressions (CTEs) and SQL `@Relation` annotations for prerequisite graph lookups.

Concurrency Model: Kotlin Coroutines & `StateFlow` for dynamic state stream evaluation and off-thread graph processing.

6. CROSS-PLATFORM FEATURE PARITY MATRIX

Feature Subsystem	Native iOS Technical Component	Native Android Technical Component
UI Engine	SwiftUI Views + Swift Charts	Jetpack Compose + Material3 Components
Local Persistence	SwiftData / CoreData Framework	Room ORM + SQLite CTE Query Engine
Graph Calculation	Swift Actor-isolated Traversal Core	Kotlin Coroutine Dispatchers.Default Engine
State Management	Observable Object / `@Observable` Macro	StateFlow / SharedFlow Event Pipeline
Haptic Feedback	`UIImpactFeedbackGenerator`	`HapticFeedbackConstants` Engine

7. SYSTEM DATA PIPELINE SEQUENCE



8. USER EXPERIENCE & INTERFACE GUIDELINES

The mobile experience is centered on transparency—demystifying complex prerequisite structures through clean, human-centered UI patterns.

Interactive Prerequisite Visualizer

A pan-and-zoom interactive graph view allowing students to expand course nodes and inspect dependency paths. Missing prerequisites are highlighted with distinct status warnings.

Confidence Rating Controls

Intuitive subject sliders allowing students to grade their confidence (1-10) in foundational areas (e.g., Calculus, Physics, Coding). Real-time visual meters reflect predicted workload stress.

Semester Stress Meter

A real-time gauge component driven by the Fuzzy Engine output. Color-coded indicators (Green: Balanced, Amber: Challenging, Red: Overload Risk) inform students before finalizing registration lists.

Transparent Reasoning Cards

Provides human-readable context for course placements (e.g., *"MATH201 recommended for Term 2 to unlock CS301 by Junior Year"*).

9. TECHNICAL EDGE CASE HANDLING & SECURITY

- **Circular Prerequisite Prevention:** If a curriculum file contains a loop (e.g., Course A requires B, and B requires A), the graph validator catches the cycle during import, halts traversal, and alerts the user without throwing a stack overflow exception.
- **Data Integrity & Transcript Privacy:** All student transcript records, grades, and confidence parameters remain fully encrypted on-device using platform keystores (iOS Keychain / Android KeyStore). No user educational data is sent to external servers.
- **Unsatisfiable Goals:** If a student attempts to schedule a target major in fewer terms than physically required by the longest path (Critical Path Length) in the DAG, the app generates an "Extended Term Required" resolution prompt.

10. QUALITY ASSURANCE & TESTING STRATEGY

Algorithmic Unit Tests

100% test coverage on Graph Traversal and Fuzzy Rule Engine modules using synthetic test catalogs across edge-case DAG topologies.

Deterministic Benchmarks

Performance profiling ensuring graph execution finishes in < 16ms to maintain 60 FPS / 120 FPS UI scroll responsiveness during recalculations.

UI & Flow Validation

Automated XCTest / Jetpack Compose UI tests verifying drag-and-drop course reordering and slider state updates.